Aprendizaje No Supervisado

K-means, clustering jerárquico y PCA

Analítica de Datos, Universidad de San Andrés



Hasta ahora: aprendizaje supervisado

Supervisado (CART, KNN, RF, Naive Bayes)

- Tenemos una columna respuesta: ¿pagó?, ¿compró?, ¿precio?
- El modelo aprende a **predecirla**
- Podemos medir si acierta (accuracy, ROC, ...)

No supervisado (hoy)

- No hay respuesta: solo tenemos las características
- Buscamos **estructura**: grupos, patrones, resúmenes
- No hay "acierto": la utilidad la decide el negocio

"No sé qué tipos de clientes tengo" vs. *"Sé quién se fue, quiero predecir quién se va a ir"*

Tres preguntas de negocio, tres técnicas

Pregunta	Técnica	Demo de hoy
¿Qué tipos de clientes tengo? ¿A quién le mando qué campaña?	**K-means**	200 clientes de un shopping
¿Qué países se parecen entre sí para expandirme?	**Clustering jerárquico**	193 países (ONU)
Tengo 12 métricas que dicen lo mismo: ¿cómo las resumo en un mapa?	**PCA**	93 modelos de autos

Las dos primeras **agrupan filas** (clientes, países). PCA **resume columnas** (variables).

K-means: segmentar clientes

El problema



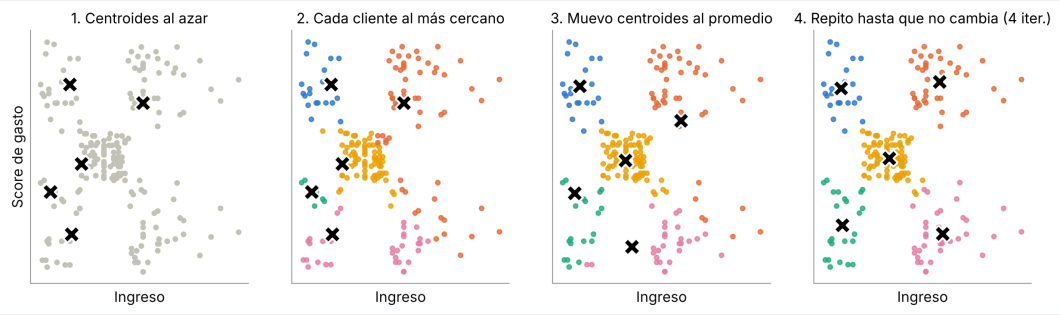
Marketing de un shopping. Por cada cliente de la tarjeta de fidelidad tenemos:

- Ingreso anual
- Score de gasto (1–100)

Queremos armar campañas para distintos tipos de clientes, pero nadie nos dijo cuáles son.

¿Cuántos grupos ves?

La idea: K “clientes representantes”



Elegimos **K** y tiramos K centroides al azar. Cada cliente se asigna al centroide más cercano, cada centroide se mueve al promedio de sus clientes, y repetimos hasta que nada cambia.

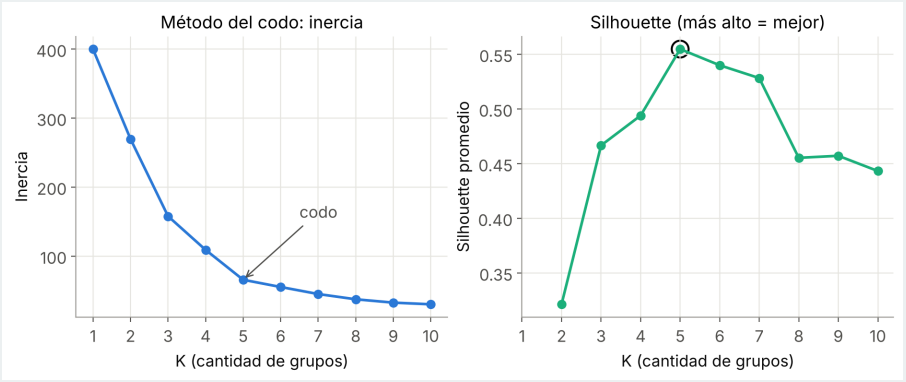
La matemática, muy por arriba

K-means busca los grupos que minimizan la inercia:

$$\text{Inercia} = \sum_{\text{grupos}} \sum_{\text{clientes del grupo}} \text{distancia}(\text{cliente}, \text{centroide})^2$$

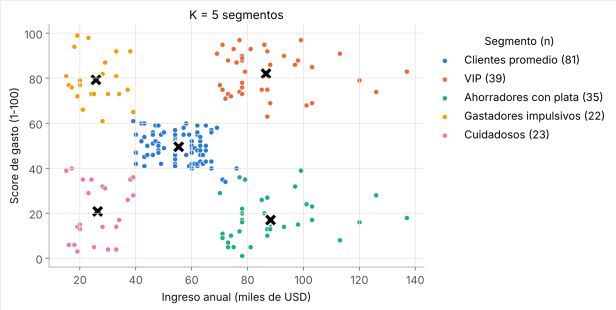
- Traducción: **grupos lo más "apretados" posible** alrededor de su representante.
- "Distancia" = distancia en línea recta (euclídea) entre los valores de las variables.
- Por eso hay que estandarizar: si el ingreso está en pesos y el score en 1–100, el ingreso domina la distancia.
- El resultado depende del arranque al azar $\Rightarrow$ se corre varias veces y se queda con la mejor (`n_init=10`).

¿Cuántos grupos? K lo elegimos nosotros



- **Codo**: la inercia siempre baja al agregar grupos; buscamos donde *deja de bajar mucho*.
- **Silhouette**: ¿cada cliente está más cerca de su grupo que del grupo vecino? (de -1 a 1).
- Y una pregunta de negocio: ¿cuántas campañas distintas puede manejar el equipo?

El algoritmo da números; el nombre lo ponemos nosotros



Segmento	Acción
VIP	Fidelizar, eventos exclusivos
Ahorradores con plata	Mayor potencial: ¿por qué no compran acá?
Gastadores impulsivos	Promos, cuotas; son los más jóvenes
Cuidadosos	Descuentos puntuales
Promedio	Comunicación masiva

Cuidado con K-means

- **Siempre devuelve K grupos**, aunque los datos no tengan grupos.
- **Estandarizar** las variables antes (trabaja con distancias).
- Le gustan los grupos **redondos y de tamaño parecido**; con formas raras falla.
- **Outliers** tiran de los centroides.
- Solo variables **numéricas** (las categóricas hay que codificarlas con cuidado).
- Escala muy bien: funciona con millones de clientes.

Demo 1: segmentación de clientes

`01_kmeans_clientes.ipynb`

Clustering jerárquico: agrupar países

El problema

Una **fintech argentina** quiere expandirse. Antes de estudiar mercado por mercado:

¿Qué países se parecen entre sí?

- 193 países, 5 indicadores de la ONU: fertilidad, PBI per cápita, esperanza de vida, % urbano, mortalidad infantil.
- No sabemos cuántos grupos hay y queremos algo fácil de mostrarle al directorio.
- Si una estrategia funciona en un país, probablemente funcione en sus "vecinos".

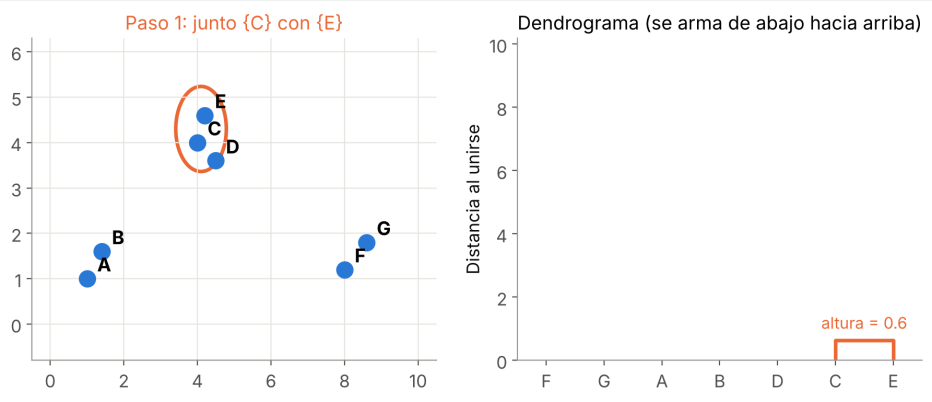
La idea en una frase

Arranco con cada país solo y voy juntando de a dos
a los más parecidos, hasta que queda un solo grupo.

- Es como armar un **árbol genealógico** de parecidos: primero se juntan los "hermanos", después los "primos", y así.
- El **dendrograma** es el dibujo de ese árbol: anota **quién** se juntó con quién y **qué tan distintos** eran cuando se juntaron.
- No hace falta elegir K antes: la cantidad de grupos se decide **al final**, mirando el árbol.

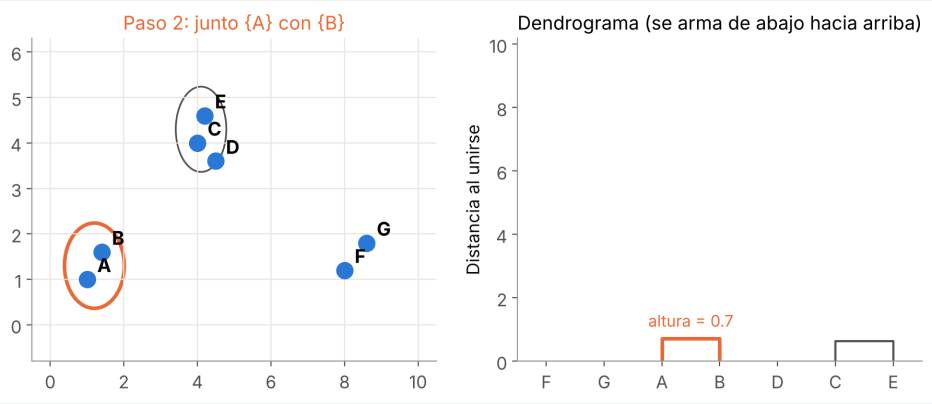
Veámoslo con 7 puntos, paso por paso $\rightarrow$

Paso a paso: cómo se arma el árbol



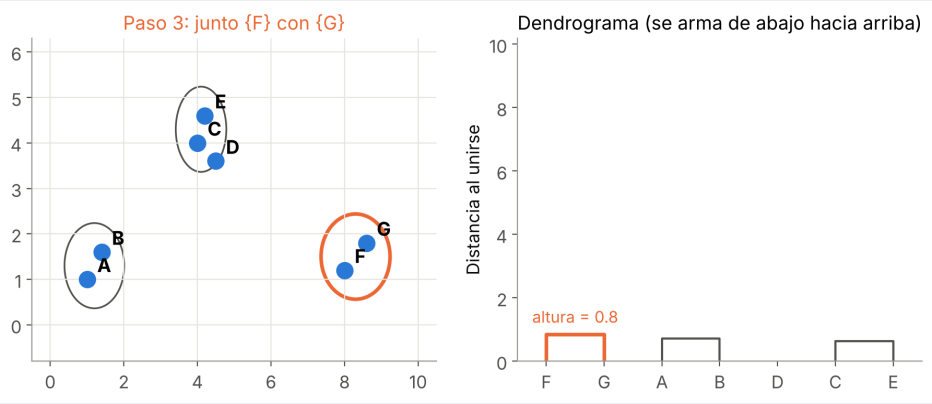
Los dos puntos **más cercanos** (C y E) se juntan primero. En el árbol aparece una "U" **bajita**: eran muy parecidos.

Paso a paso: cómo se arma el árbol



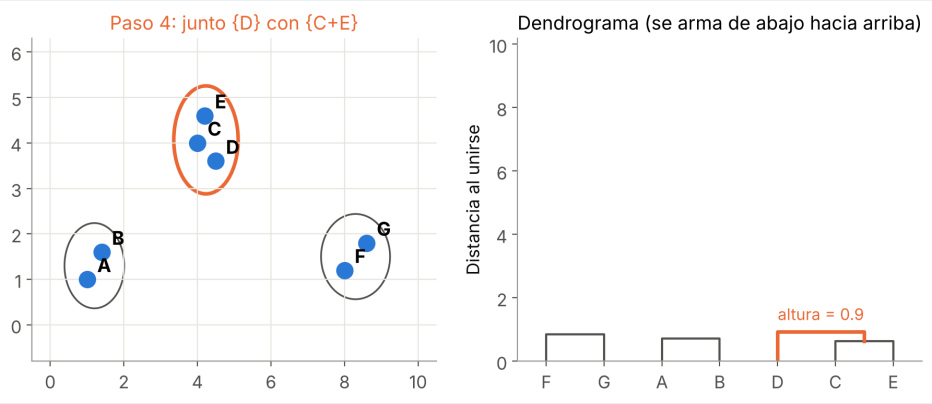
Siguiente par más cercano: A y B. Otra “U” bajita.

Paso a paso: cómo se arma el árbol



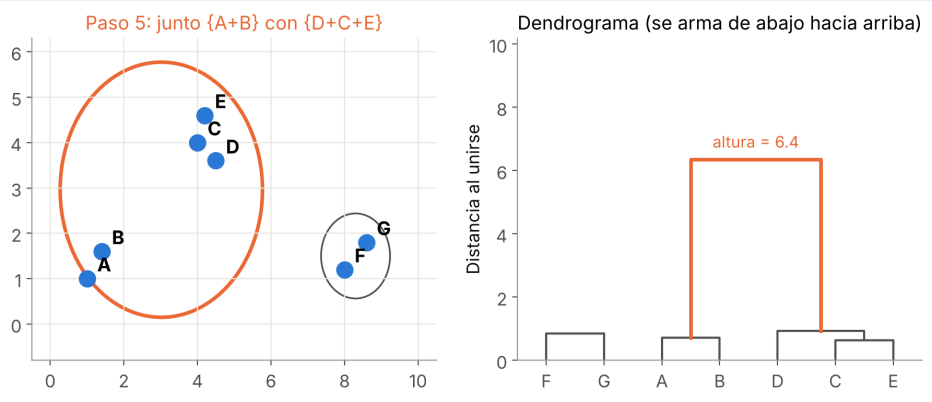
F y G. Ya tenemos 3 grupitos bien apretados.

Paso a paso: cómo se arma el árbol



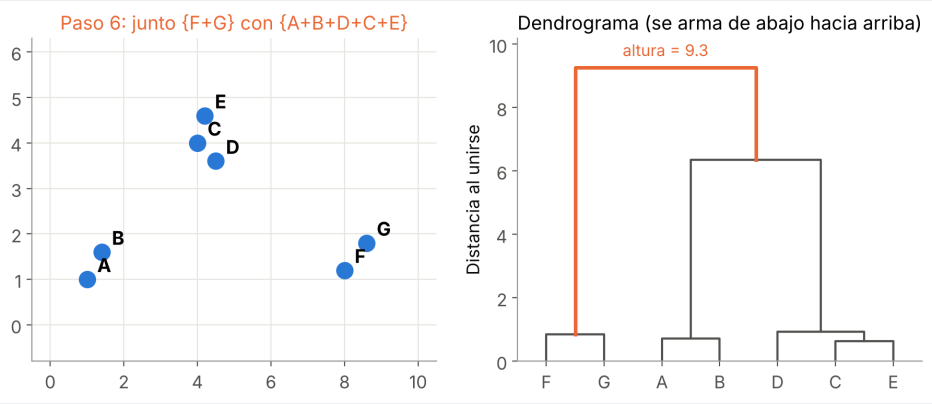
D se suma al grupo C, E. Ahora quedan 3 grupos: A,B, C,D,E y F,G.

Paso a paso: cómo se arma el árbol



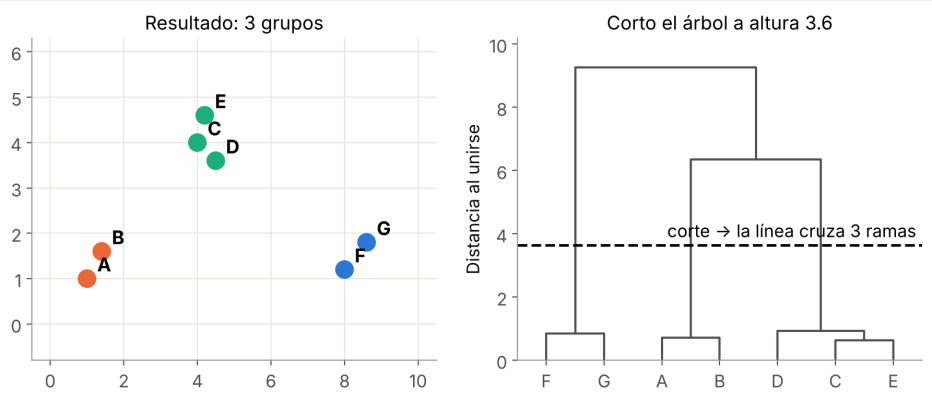
Ahora sí hay que juntar grupos lejanos: la "U" sube mucho (altura 6.4). Ese **salto** es la señal de que estamos mezclando cosas distintas.

Paso a paso: cómo se arma el árbol



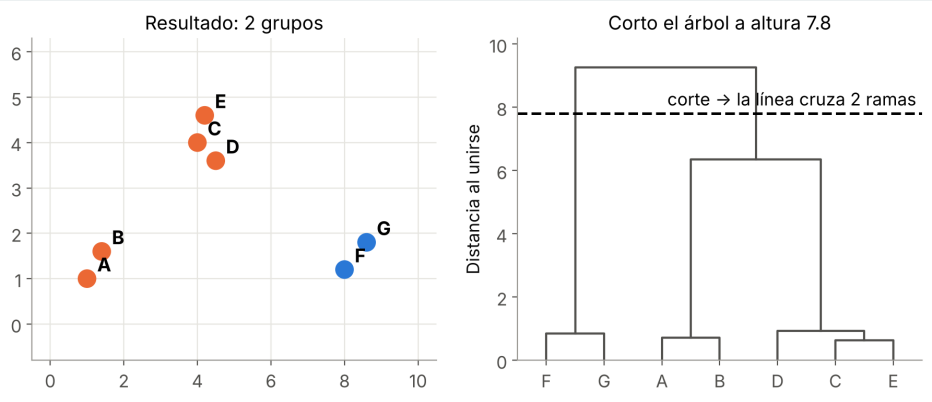
Última unión: todo queda en un solo grupo. El árbol está completo.

¿Cuántos grupos? Corto el árbol



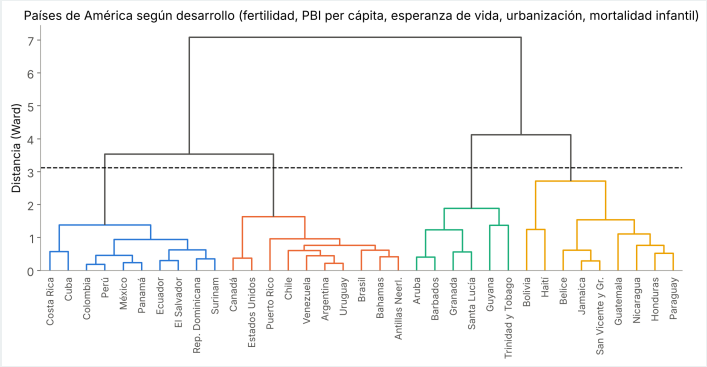
Trazo una **línea horizontal**: la cantidad de ramas que corta = la cantidad de grupos. Cortando por **debajo del gran salto** quedan 3 grupos que tienen sentido.

¿Cuántos grupos? Corto el árbol



Trazo una **línea horizontal**: la cantidad de ramas que corta = la cantidad de grupos. Si corto más arriba, quedan 2 grupos más "gruesos". ¿Cuál conviene? Depende de la pregunta de negocio.

Ahora con datos reales: países de América



- Argentina y Uruguay se juntan **muy abajo**: son muy parecidos en estos indicadores.
- El salto grande está alrededor de 3: si corto ahí (línea punteada) quedan 4 grupos de países.

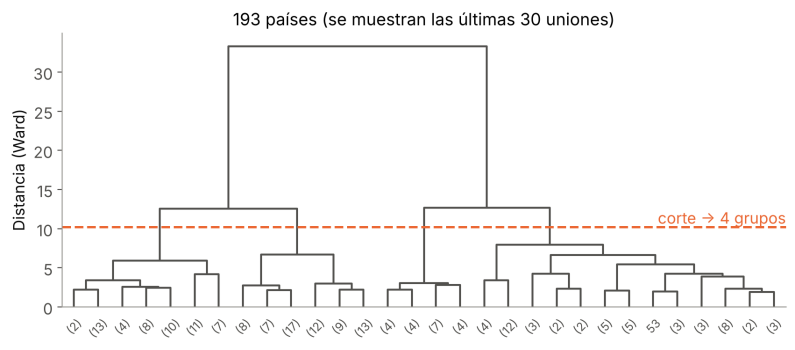
La matemática, muy por arriba: ¿cómo mido la distancia entre grupos?

Entre dos países es fácil: la misma distancia en línea recta que en K-means. Pero cuando ya tengo **grupos**, hay que decidir cómo compararlos. A eso se le dice linkage:

single	los dos miembros **más cercanos** → arma "cadenas", no sirve para segmentar
complete	los dos miembros **más lejanos**
average	el **promedio** de todas las distancias
ward	junta los grupos que quedan **más apretados** (como K-means)

En la práctica: **usamos Ward**. En la demo se ve qué pasa con los otros.

Con todos los países



Grupo (mediana)	**Países**	**PBI p/c (USD)**	**Esp. vida**	**Hijos/mujer**
Alto desarrollo (incluye Argentina)	66	20.556	80,7	1,9
Desarrollo medio-alto	55	4.496	77,3	2,0
Desarrollo medio-bajo	53	1.240	64,4	3,8
Bajo desarrollo	19	451	53,1	5,5

K-means vs. jerárquico

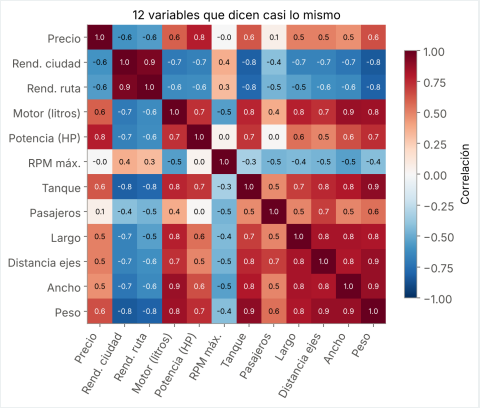
	K-means	**Jerárquico**
¿Elijo K antes?	Sí	No: corto el árbol después
Resultado	Grupos + centroides	Árbol completo (dendrograma)
Cantidad de datos	Millones, sin problema	Hasta miles (es lento)
Explicarlo	"Estos son los 5 perfiles"	"Mirá quién se parece a quién"
¿Mismo resultado si lo corro de nuevo?	Depende del arranque	Sí, siempre

Demo 2: ¿qué países se parecen?

`02_jerarquico_paises.ipynb`

PCA: resumir muchas variables

El problema

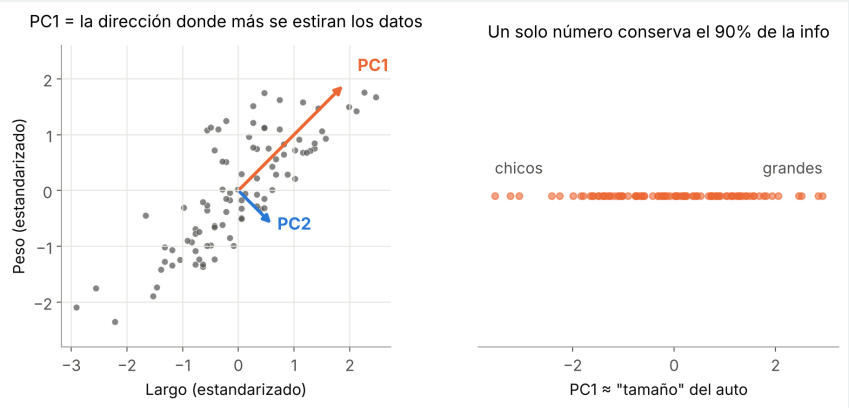


Trabajamos en una automotriz. Queremos ver **cómo están posicionados** los 93 modelos del mercado: ¿dónde hay competencia?, ¿hay huecos?

Tenemos 12 características por modelo, pero:

- No se pueden graficar 12 dimensiones.
- Muchas dicen lo mismo: peso, largo, ancho, motor... todas miden "qué tan grande es".

La idea: buscar la dirección donde los datos más varían



Largo y peso van juntos. Si giro los ejes y miro solo **PC1**, con **un número** conservo casi toda la información. Ese número es, en la práctica, el "tamaño" del auto.

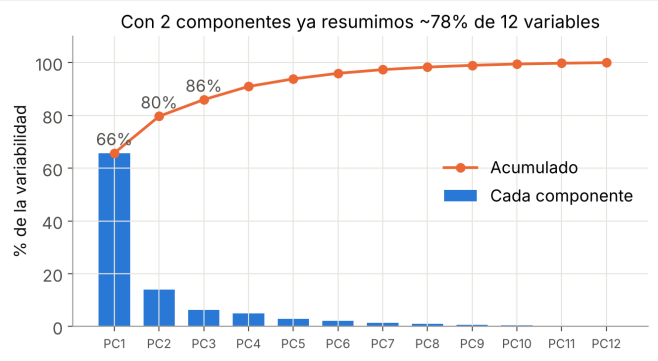
La matemática, muy por arriba

Cada componente principal es una receta: una suma ponderada de las variables originales (estandarizadas).

$$\text{PC1} = w_1 \cdot \text{precio} + w_2 \cdot \text{rendimiento} + \cdots + w_{12} \cdot \text{peso}$$

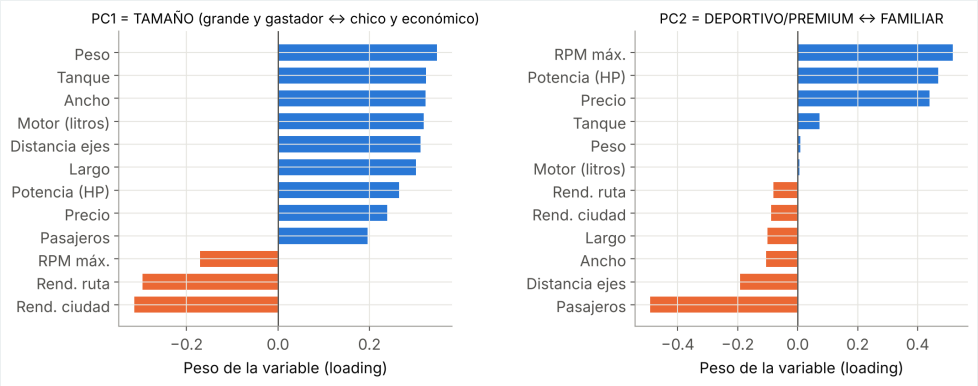
- PCA elige los pesos w (**loadings**) para que PC1 tenga la **mayor varianza posible**.
- PC2 hace lo mismo con lo que sobra, **sin repetir** información de PC1 (son perpendiculares). Y así sucesivamente.
- Hay tantas componentes como variables, pero las primeras concentran casi todo.
- Igual que en clustering: estandarizar antes, si no gana la variable con números más grandes.

¿Cuántas componentes me quedo?



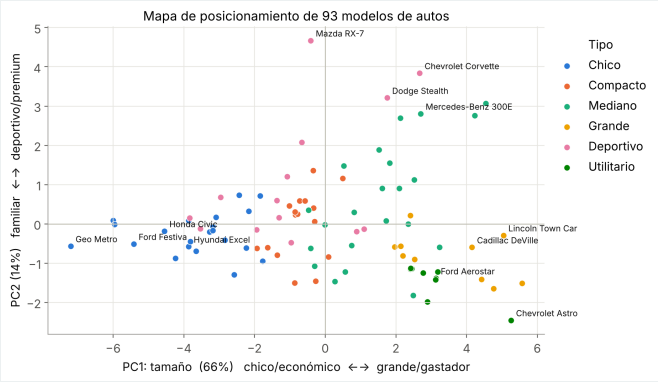
- Mirar el **acumulado**: ¿con cuántas llego a 80–90 %?
- Para **visualizar**: 2 (o 3). Para **modelar**: las que hagan falta.

PCA no le pone nombre: lo hacemos nosotros



Mirando los loadings: **PC1 = tamaño** (todo lo "grande" suma, el rendimiento resta). **PC2 = deportivo/premium vs. familiar** (potencia y precio suman, pasajeros resta).

El resultado: un mapa de posicionamiento



- El **tipo de auto no se usó**, y aun así queda ordenado: las componentes capturan algo real.
- Vecinos en el mapa = **competidores directos**.
- ¿Huecos? Grande *y* deportivo: casi vacío.

Cuidado con PCA

- **Estandarizar** antes.
- Las componentes son **difíciles de explicar**: "subió PC3" no le dice nada a nadie. En un modelo predictivo perdés explicabilidad.
- Solo captura relaciones **lineales**.
- Solo variables numéricas.
- Muy útil para **visualizar**, armar **índices** (ej. "índice de desarrollo") y como paso previo a clustering.

Demo 3: mapa de posicionamiento

`03_pca_autos.ipynb`

Resumen

	K-means	**Jerárquico**	**PCA**
Hace	Agrupa filas en K grupos	Arma un árbol de parecidos	Resume columnas
Usalo para	Segmentar muchos clientes	Pocos casos, sin K claro	Mapas, índices, quitar redundancia
Decidís vos	K	Dónde cortar, linkage	Cuántas componentes
Siempre	Estandarizar · interpretar y ponerle nombre · validar con el negocio		

Se combinan: PCA para resumir → clustering sobre las componentes (bonus en la demo 3).

Terminamos

¿Dudas?

¿Consultas?

